

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄奕皓 学号 2024K8009929012 专业 计算机科学与技术
实验项目编号 4.3 实验名称 深度学习算法与硬件加速器

注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。

注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。

注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、硬件与软件设计实现

1.1 ALU 模块扩展与乘法指令实现

为支持乘法指令, 本次实验将 ALU 的操作码由原来的 3 位扩展到 4 位。原有所有指令的操作码在高位补 0, 功能保持不变, 确保了向后兼容。新增的乘法指令(MUL)对应的操作码为 4'b1000。该指令完成两个 32 位有符号数的乘法, 取乘积的低 32 位作为结果写入通用寄存器, 符合 MIPS/RISC-V 架构中 MUL 指令的行为定义。运算结果直接通过 ALU 主输出通路送往写回阶段, 无需独立的 HI/LO 寄存器。

在数据通路上, ALU 位于处理器的执行阶段, 两个输入分别来自寄存器堆的源操作数或立即数扩展结果; 控制信号为 4 位操作码, 由译码模块根据指令 opcode 生成。乘法运算复用了与加减、逻辑运算相同的数据通路, 未增加额外连线。ALU 内部各运算结果并行计算, 最后由操作码选择输出。以下代码片段展示了 ALU 中基于操作码的结果选择逻辑, 与实际的硬件描述完全一致:

Listing 1: ALU 结果选择逻辑

```
assign Result = (ALUop == 4'b0000) ? and_result :  
    (ALUop == 4'b0001) ? or_result :  
    (ALUop == 4'b0100) ? xor_result :  
    (ALUop == 4'b0101) ? nor_result :  
    (ALUop == 4'b0010) ? add_result[31:0] :  
    (ALUop == 4'b0110) ? add_result[31:0] :  
    (ALUop == 4'b0111) ? slt_result :  
    (ALUop == 4'b0011) ? sltu_result :  
    (ALUop == 4'b1000) ? mul_result :  
    `DATA_WIDTH'd0;
```

其中 4'b0010 为加法(ADD), 4'b0110 为减法(SUB), 4'b0111 为有符号比较(SLT), 4'b0011 为无符号比较(SLTU), 逻辑运算分别对应 AND、OR、XOR、NOR。乘法指令 4'b1000 则直接输出有符号乘积的低 32 位。

1.2 深度学习软件算法的设计与实现

定点数表示与运算策略 依据讲义 §2.2 的定义, 输入图像、卷积核权重以及输出特征图均采用 16 位 Q5.10 定点格式(1 位符号、5 位整数、10 位小数), 在 C 语言中对应 short 类型。其具体位段划分见表 1。

表 1: Q5.10 定点数位段划分

位段	符号位	整数位	小数位
位宽	1 bit	5 bits	10 bits
位置	第 15 位	第 14~10 位	第 9~0 位

为避免溢出与精度损失, 软件实现采取了如下策略: 两个 Q5.10 数相乘后得到 Q10.20 格式的中间结果, 使用 32 位有符号整数 int 作为累加器, 完整保留全部中间精度; 对一个输出像素的所有乘加运算及偏置叠加完成之后, 才统一进行右移 10 位的归一化操作, 将结果还原为 Q5.10 格式, 避免了多次移位带来的累积截断误差; 最终写入结果前, 再进行饱和截断, 限制在 short 的有效范围 $[-32768, 32767]$ 内。

内存地址映射

实验所涉及的各种数据与设备在系统地址空间中的布局如表 2 所示。所有数据均通过链接脚本预加载到对应地址, 程序运行时直接通过指针对其进行访问。软件视角下卷积核按「1 个 bias + 25 个权重」紧密排列(每个卷积核 52 字节); 硬件视角下会额外填充空白实现 64 字节对齐, 因此 comparing() 函数在硬件模式下会跳过每行末尾的对齐填充字节, 仅比对有效数据。

表 2: 系统内存与外设地址映射

数据/外设类别	起始地址	说明
输入图像	0x080d8f90	单通道 28×28 Q5.10 定点特征图
卷积核权重	0x08000000	20 组 5×5 卷积核, 每组含 1 个偏置 +25 个权重
金标准输出	0x07000000	预置的正确结果, 用于比对
算法输出	0x080d9690	卷积 + 池化后的特征图写入区域
DNN 加速器	0x60030000	MMIO 外设基地址, 含 START、DONE 寄存器
性能计数器	0x60010000	周期计数器寄存器地址

2D 卷积函数实现

卷积的输入为单通道 28×28 的特征图, 使用 20 组 5×5 卷积核, 无 padding, 步长为 1, 输出为 20 通道、每通道 24×24 的特征图。输出尺寸计算依据下式:

$$o_w = \left\lfloor \frac{i_w - k_w + 2 \times pad}{stride} \right\rfloor + 1$$

内存布局采用 NCHW(通道优先)格式, 各通道间紧密排列; 每个卷积核在内存中按“1 个 short 型 bias + 25 个 short 型权重”的方式连续存放。计算时最外层遍历输出通道、行、列, 内层在卷积核窗口内遍历行、列。当坐标超出输入图像边界时, 自动取填充值 0 参与运算。核心的乘加与归一化过程如下:

Listing 2: 卷积乘加与归一化核心代码

```
int acc = 0;
for (unsigned kh = 0; kh < weight_size.d2; kh++) {
    int ih = (int)(oh * stride) + kh - (int)pad;
    for (unsigned kw = 0; kw < weight_size.d3; kw++) {
        int iw = (int)(ow * stride) + kw - (int)pad;
        short pixel_val = 0;
        if (ih >= 0 && ih < (int)input_fm_h &&
```

```

        iw >= 0 && iw < (int)input_fm_w) {
            pixel_val = in[ih * input_fm_w + iw];
        }
        short w_val = weight[weight_base + 1 + kh * weight_size.d3 + kw];
        acc += mul(pixel_val, w_val);
    }
}
acc += (int)bias << FRAC_BIT;
acc >>= FRAC_BIT;
if (acc > 32767) acc = 32767;
else if (acc < -32768) acc = -32768;

```

最大值池化函数实现

池化采用 2×2 窗口，步长为 2，无 padding。输入为卷积输出的 20 通道 24×24 特征图，输出为 20 通道 12×12 特征图。函数采用原地覆盖策略，输出直接写回原卷积输出所在的内存区域，节省了额外的缓冲区。边界处理上，仅对窗口内实际位于输入范围内的像素取最大值，越界位置直接忽略，从而避免了无效 0 值可能引起的错误比较。池化输出尺寸除标准公式外，额外处理了无 padding 且尺寸无法被步长整除的边界场景，保证最后一行 / 列的不完整窗口也会被采样。

硬件加速器控制

依据讲义 §2.5 的说明，DNN 加速器作为一个 MMIO 外设挂载在系统总线上，处理器通过 Load/Store 指令访问其控制寄存器。加速器基地址为 0x60030000，偏移 0x0 的 START 寄存器用于启动运算（向其第 0 位写 1），偏移 0x8 的 DONE 寄存器用于查询完成状态（第 0 位为 1 表示运算结束）。软件中的控制流程非常简单：向 START 写入启动命令后，通过 volatile 指针循环读取 DONE 寄存器，直至其第 0 位变为 1。关键实现如下：

Listing 3: 硬件加速器启动与轮询

```

void launch_hw_accel()
{
    volatile int *gpio_start = (volatile int *) (GPIO_START_ADDR);
    volatile int *gpio_done = (volatile int *) (GPIO_DONE_ADDR);
    *gpio_start = 0x1;
    while ((*gpio_done & 0x1) == 0);
}

```

性能统计与结果校验

利用 perf_cnt 接口中的 bench_prepare 与 bench_done，在卷积和池化函数调用前后分别插入计数器读取，从而单独统计两个阶段的执行周期。结果校验由 comparing() 函数完成，它将内存中的输出特征图与金标准二进制文件逐字节比较，并输出“Passed”或“Failed”信息。

此外，代码通过编译宏区分三种运行模式：无特殊宏定义为纯软件模拟乘法；定义 USE_MUL 则启用硬件乘法指令；定义 USE_HW_ACCEL 则完全交由硬件加速器处理。mul() 函数通过 USE_MUL 宏切换实现：未定义时调用 mul_ll() 软件模拟乘法，定义时直接使用 * 运算符，由编译器生成硬件 MUL 指令。

实测性能对比

三种实现方案均已在 FPGA 上通过了功能验证（输出 Passed!），其总运行时间与主要特性汇总于表 3。

从数据中可以得出：

- 引入硬件乘法指令后，整体加速比约为 $\frac{155766.94}{3419.61} \approx 45.6$ 倍，充分体现了指令集层面的优化对计算密集型任务的巨大改善；
- 使用专用硬件加速器后，相较于纯软件实现，加速比达到 $\frac{155766.94}{61.25} \approx 2543$ 倍；即使与带硬件乘法的软件版

表 3: 三种实现方案的性能对比

实现方案	总运行时间	功能验证	核心说明
纯软件实现(软件模拟乘法)	155766.94 ms	通过	无硬件乘法指令,通过 mul_ll 函数逐位模拟乘法
软件 + 硬件乘法指令	3419.61 ms	通过	定义 USE_MUL, 调用处理器的 MUL 指令完成单周期乘法
专用硬件加速器	61.25 ms	通过	定义 USE_HW_ACCEL, DMA 自动读写内存并完成全部运算

本相比, 仍有约 $\frac{3419.61}{61.25} \approx 55.8$ 倍的提升。这主要得益于运算并行化、数据预取以及专用数据通路带来的协同加速效应。

1.3 实验总结

本实验沿着“纯软件实现 → 指令集优化 → 硬件加速器加速”的路径, 完整地展现了计算机系统从软件层到硬件层的多层次优化策略。在算法层面, 通过定点数替代浮点数、全精度累加与单次归一化等手段, 在保证计算精度的同时有效实现了 2D 卷积与最大值池化; 在指令集层面, 仅增加一条乘法指令就使整体性能提升了数十倍; 在硬件加速层面, 通过专用 DNN 加速器将繁重的计算完全卸载到硬件, 处理器仅负责启停控制, 最终获得了三个数量级的性能飞跃。实验结果印证了体系结构设计中的重要思想: 对于特定领域的计算负载, 专用硬件能够带来远超通用处理器指令级优化的性能收益, 而将两者合理结合, 则可实现性能的阶梯式跃升。

二、实验过程中遇到的问题与解决

2.1 编译未使用变量错误

编译框架中预留了 output_offset 与 input_offset 两个局部变量, 但实际算法实现并未用到它们。由于编译选项包含 -Wall -Werror, 未使用变量的警告被升级为错误, 导致软件编译失败。为了尽量保持模板原有结构, 我们没有直接删除变量, 而是在声明之后添加了 (void)output_offset; 和 (void)input_offset; 两条语句, 显式向编译器表明这些变量已被引用, 从而消除了警告, 编译顺利通过。

2.2 池化边界处理逻辑错误

最初实现的池化函数中, 当窗口部分超出输入边界时, 对应位置的像素值被设为 0 并参与最大值比较。然而, 当窗口内所有有效像素均为负数时, 0 会被错误地选为最大值, 导致结果与金标准不符。分析后意识到, Max Pooling 的语义是对窗口覆盖的真实像素取最大值, 越界位置并不属于特征图的一部分, 不应纳入比较。因此, 我们修改了判断逻辑: 只在坐标位于有效范围内时才读取像素并更新最大值, 越界处直接跳过。修正后所有边界位置的池化结果均与金标准完全一致。

2.3 定点数归一化时机的精度问题

初期实现中, 每次乘法后都立即对结果进行右移归一化再累加, 这造成了多次低位截断, 累积误差随乘加次数增加而增大, 最终导致输出与金标准存在偏差。经过思考, 我们将策略调整为全精度累加: 所有乘加结果以 Q10.20 格式暂存在 32 位累加器中, 待一个输出像素的所有运算(包括偏置叠加)完成后, 才统一进行右移 10 位的归一化。这一做法最大限度地保留了中间精度, 使最终结果与金标准完全吻合。

三、 对讲义中思考题的理解与回答

思考题:UART 驱动与硬件寄存器访问代码中,volatile 关键字的作用是什么? 如果去掉会出现什么后果?

volatile 是 C 语言中的类型限定符,它告诉编译器:被修饰的变量可能被硬件外设、中断服务程序或其他执行上下文异步修改,因此禁止对该变量的访问进行任何优化。具体地,编译器必须保证每次读写都直接访问内存地址,不得将变量缓存在寄存器中,也不得合并或删除看似冗余的访问指令,同时要维持代码书写的访问顺序,不进行指令重排。

如果去掉 volatile,在轮询硬件状态的场景中(例如等待 UART 发送队列就绪,或等待加速器完成标志),编译器可能会将状态寄存器的值读取一次后缓存到寄存器中,之后每次循环都直接使用该缓存值,导致程序永远观察不到硬件状态的变化,陷入死循环。同样,对于向硬件控制寄存器写入的命令,编译器可能判定为“无副作用”而将其优化掉,使硬件无法接收到有效的控制信号。此外,编译器还可能调整读写顺序,破坏硬件所需的访问时序,引发不可预知的错误。因此,所有指向 MMIO 硬件寄存器的指针都必须使用 volatile 限定,这是编写外设驱动时必须遵守的基本准则。

四、 实验所耗时间

在课后,你花费了大约 3 小时完成此次实验。